Build, Test & Flash

Configuring, building, testing and flashing both targets

Two build trees

CMake caches every `-D` option, so an option stays set until you pass it again. There are two separate build directories.

Host build (GTest unit tests)

```
cmake -S . -B build-host            # configure (BUILD_STM32 defaults to OFF)
cmake --build build-host            # build app_logic + every test executable
```

Command	What it does
`cmake --build build-host --target run_tests`	Run every suite through ctest
`N=<regex> ... --target run_tests`	Run only tests matching a ctest `-R` regex, e.g. `N=HashTableTest`
`cmake --build build-host --target test_help`	List every test executable and its tests
`N=<Suite.Name> F=<file> ... --target test_debug`	Run one test under gdb
`ctest --test-dir build-host -C Debug [-V]`	Plain ctest

Nine test executables (`tests/CMakeLists.txt`): `test_hash_table`, `test_free_list`, `test_usage_bitmap`, `test_journal`, `test_ring_buffer`, `test_sector_crc`, `test_db_recovery`, `test_message_history`, `test_call_history` – 336 tests total across 14 GTest suites. See **04 – Project Status & Next Steps** for what's recommended to add next, and `notes/unit_tests/README.md` for a suite-by-suite breakdown.

STM32 build (on-hardware)

```
cmake -S . -B build -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE="firmware/cmake/gcc-arm-none-eabi.cmake" -DBUILD_STM32=ON
cmake --build build
```

The Ninja generator and the toolchain file are both required – configuring `BUILD_STM32=ON` without them stops with a usage error. Target: `STM32H723ZGTx` (LQFP144), configured in `firmware/firmware.ioc` (CRC, FATFS, FMC, FreeRTOS, SDMMC1).

Build option flow

`main.c` picks the first option set, in this order: `SD_BUS_TEST`, `PERSIST_DEMO`, `MH_TEST`, `MH_DEMO`, `SA_TEST`, `DATABASE_TEST`, `DB_DEMO`, `NAME_INDEX_BENCH`, else `db_main.c`'s default hardware DB unit tests. `MH_TEST`, `MH_DEMO` and `PERSIST_DEMO` all use the start of the message history region, so each one wipes the others' history data.

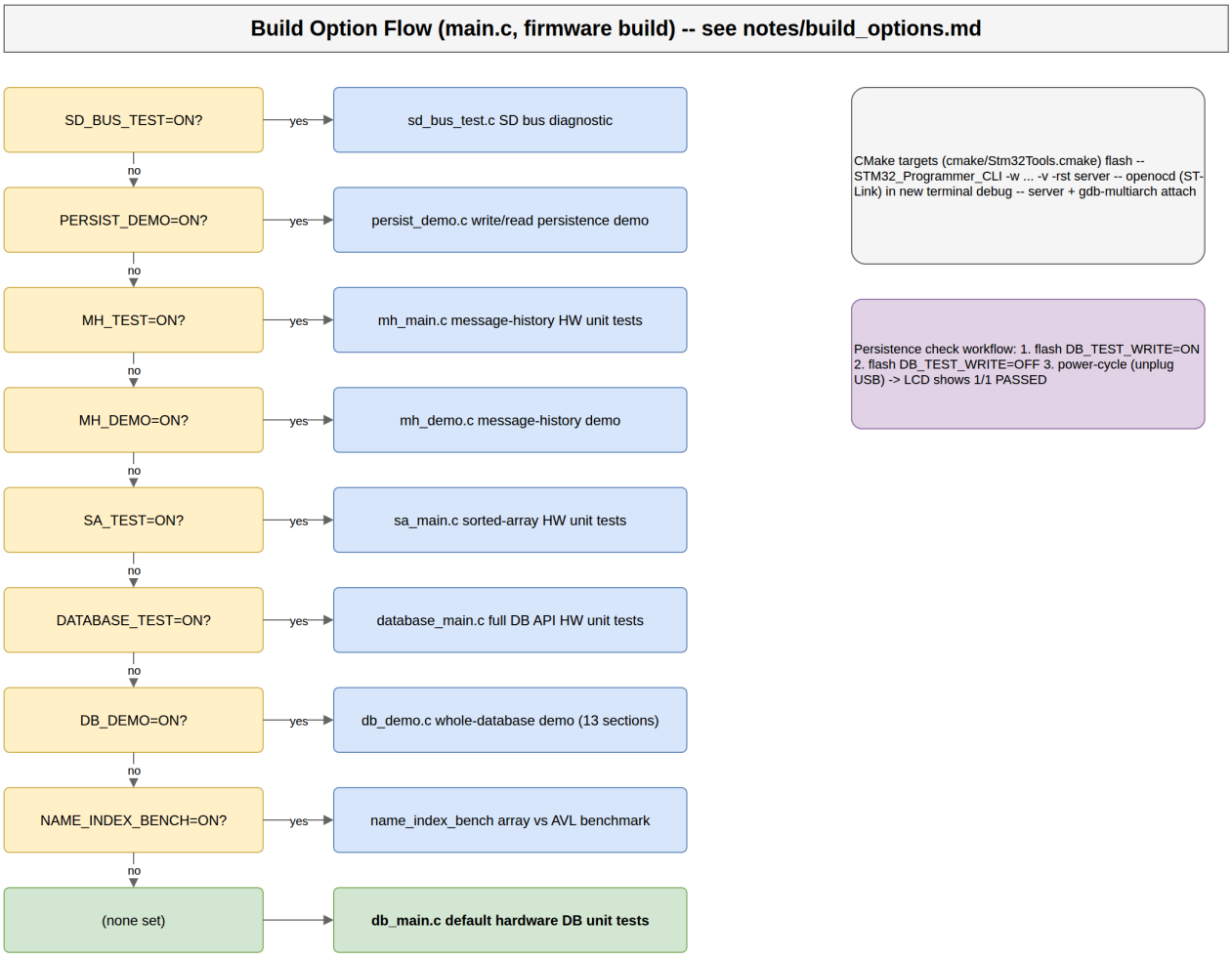


Figure 1: Build option precedence and the CMake `flash` / `server` / `debug` targets. Full option table and default values in `notes/build_options.md`.

Flashing and debugging

Target	What it does
`cmake --build build --target flash`	Build, then flash over SWD with `STM32_Programmer_CLI` (verify + reset). Uses `sudo`
`cmake --build build --target server`	Start an OpenOCD (ST-Link) server in a new terminal
`cmake --build build --target debug`	Start the server, then attach `gdb-multiarch` to `localhost:3333`

Flashing without `sudo`:

```
STM32_Programmer_CLI -c port=SWD -w build/firmware/MyProject.elf -v -rst
```

No dedicated OpenOCD `.cfg` file is checked in – `server` invokes the system-installed OpenOCD with its stock `interface/stlink.cfg` and `target/stm32h7x.cfg`.

Reading results off the board

- **LCD** – live test list and pass/fail summary (`htest_ui.c`). The demos (`persist_demo.c`, `mh_demo.c`) draw a scrolling record list (`lcd_log.c`) and report only on the LCD.
- **SWD, without halting** – read `g_db_report` (or `g_mh_report` / `g_sa_report` / `g_sd_bus_report` for the matching test). Find the address with `arm-none-eabi-nm build/firmware/MyProject.elf | grep g_db_report`, then `STM32_Programmer_CLI -c port=SWD mode=HOTPLUG -r32 <addr> 0x100`. Layout is `DbTestReport` in `db_main.c`.
- **LED (PB0)** – fast blink = all passed, N pulses = test N failed first. `StartDefaultTask` also toggles PB0, so this isn't reliable at the moment.

Persistence verification workflow

The core proof that writes survive a power cycle:

1. Flash with `DB_TEST_WRITE=OFF` is the **read half** – but first write a known data set with `DB_TEST_WRITE=ON`, then rebuild read-only:

```
cmake -S . -B build -DDB_TEST_WRITE=OFF && cmake --build build --target flash
cmake -S . -B build -DDB_TEST_WRITE=ON          # switch back afterwards
```

1. Power-cycle the board (unplug USB, not just reset).
2. The LCD should show `1/1 PASSED`.

The persistence **demo** (`PERSIST_DEMO=ON`) follows the same write-half / read-half / power-cycle shape but is a visual walkthrough rather than a pass/fail test – see `notes/db_demo.md`. Its write half wipes the data set left by `DB_TEST_WRITE=ON`, and vice versa.

Both build trees

`--target misra`	cppcheck + MISRA C:2012 addon over `source/`
`--target tags`	ctags index of the project
`--target struct_layout`	Print struct layouts and static-assert results (`tools/struct_layout.py`)
`-DCMAKE_BUILD_TYPE=<type>`	Defaults to `Debug`

> Full option-by-option detail, including `SD_BUS_TEST`, `SA_TEST`, `DATABASE_TEST` and `MH_TEST`/`MH_DEMO` command sequences, lives in `notes/build_options.md` – this chapter condenses it, that file is the source of truth if the two ever disagree.